

Module 7 Week B — Stretch: Summarize-then-QA

Honors Track. This stretch is for learners who have completed all required Week B work (Reading, Drill, Lab, Integration), are On Track or Advanced, and are attending consistently. Stretch participation is graded but not required for program completion; it counts toward Honors distinction.

In this stretch you will build a small two-stage pipeline that first summarizes a long tech / entertainment news article and then runs QA on the summary. You will compare two strategies on a 20-example test set you author yourself:

- **Strategy A** — QA on the full article (using context windowing/chunking when the article exceeds the QA model's max length).
- **Strategy B** — QA on the article's summary (single short context).

You will compute EM and token-F1 on both strategies, write a memo on the trade-offs (faithfulness loss vs. compute savings vs. answer accuracy), and recommend when each strategy fits.

The pedagogical goal is to introduce **pipeline composition** — the M8 (RAG) preview where retrieval, generation, and QA chain together — and to confront the cost of summarization-induced information loss explicitly.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: Integration 7B base assignment must be substantially complete (summarization pipeline working, ROUGE harness functional, tech news corpus + reference summaries available locally).

What You Will Produce

This stretch extends your Integration 7B repo on a new branch `stretch-thu-summarize-then-qa`. New files:

1. `qa_test_set.csv` — 20 hand-authored `(qid, article_id, question, gold_answer, gold_evidence_in_article)` tuples, grounded in the tech news articles.
2. `pipeline_compose.py` — implements the two strategies (`qa_full_article`, `qa_via_summary`) and an evaluation harness over both.
3. `compose_predictions.csv` — predictions for both strategies side-by-side.
4. `compose_metrics.json` — per-strategy EM/F1.
5. `compose-analysis.md` — 1.5-page memo (5 required sections).

Setup

The stretch starter files ship inside your Integration 7B template repo at `stretch/thursday/` — they were already in the repo when you accepted the Integration 7B GitHub Classroom assignment.

```
cd <your-integration-7b-repo>
git checkout main && git pull
git checkout -b stretch-thu-summarize-then-qa
ls stretch/thursday/ # qa_test_set.csv, pipeline_compose.py, compose-analysis.md
```

You will also need the QA pipeline infrastructure from your Lab 7B work. Copy `normalize_answer`, `exact_match`, `token_f1`, `build_qa_pipeline`, `predict_one`, and `get_qa_model_name` from your Lab 7B `lab.py` into `stretch/thursday/ga_utils.py`. `pipeline_compose.py` imports from `ga_utils` and will fail clearly if it's missing. The stretch composes both pipelines — that's the whole point.

Step 1: Author the Test Set (20 examples)

Open `data/tech_news_articles.csv` (1,033-article corpus shared with the Integration 7B repo). Pick 10–15 articles whose length exceeds the QA model's max input length (~384 tokens — articles you can verify exceed this with a quick token-length check). Author 20 questions across these articles.

Constraints (the autograder enforces some):

- Exactly 20 rows.
- Each `gold_answer` must be a substring of the referenced article (extractive QA constraint).
- `gold_evidence_in_article` is a longer span (~1 sentence) containing the answer — used by the TA to verify your test-set quality.
- Mix question types: factual ("what year"), entity-attribution ("which company"), causal ("why"), and a few harder ones where the answer is in a later paragraph (relevant to chunking strategy).

Read articles carefully and pick targeted ones — the test-set design is the central engineering act of this stretch.

Step 2: Strategy A — QA on the Full Article (with Chunking)

Implement `qa_full_article(qa, question, article, max_chunk=384)`:

- If the article fits in `max_chunk` tokens, just call `predict_one(qa, question, article)`.
- If it exceeds `max_chunk`, split into overlapping chunks (e.g., 384 tokens with 64-token overlap), run `predict_one` on each chunk, and return the **highest-scoring span** across chunks.
- "Highest score" means the highest pipeline `score` field from the chunk-level call.

Why overlapping windows? Without overlap, an answer that straddles a window boundary would be missed. ~16% overlap (64 tokens out of 384) is the standard QA-windowing default.

```
def qa_full_article(qa, question: str, article: str, max_chunk: int = 384) -> str:
    """
    Run QA over the full article, chunking if it exceeds max_chunk tokens.

    Returns the best answer span across chunks (by pipeline score).
    """
    # TODO: tokenize the article to count tokens
    # TODO: if it fits, call qa once
    # TODO: if not, split into overlapping windows; call qa on each window
    # TODO: return the answer from the highest-scoring chunk
```

Step 3: Strategy B — QA on the Summary

Implement `qa_via_summary(qa, summ, question, article, max_summary_length=120)`:

- Summarize the article using your `summarize_one` from Integration 7B (with `do_sample=False`, `num_beams=4`).
- Run `predict_one(qa, question, summary)` on the summary text.

```
def qa_via_summary(qa, summ, question: str, article: str, max_summary_length: int) -> str:
    """
    Summarize first, then run QA on the summary.

    Returns the answer string.
    """
    # TODO: call summ to produce a short summary
    # TODO: call predict_one on the summary
    # TODO: return the answer
```

The autograder will static-scan that `summ` is invoked before `qa` in this function. Skipping summarization defeats the comparison.

Step 4: Evaluate Both Strategies

Implement `evaluate_strategies(qa, summ, test_set, articles_df)`:

- For each row in `test_set`, look up the article in `articles_df`.
- Run both strategies; collect predictions and per-strategy EM/F1.
- Return a dict:

```
{
    "strategy_a": {"em": float, "f1": float, "n": int},
    "strategy_b": {"em": float, "f1": float, "n": int},
    "predictions": [
        {
            "qid": str,
            "question": str,
            "strategy_a_pred": str,
            "strategy_b_pred": str,
            "gold_answer": str,
            "strategy_a_em": int,
            "strategy_a_f1": float,
            "strategy_b_em": int,
            "strategy_b_f1": float,
        },
        ...
    ],
}
```

Implement `main()` to orchestrate: load CSVs, build pipelines, run the harness, write `compose_predictions.csv` and `compose_metrics.json`.

Step 5: Write the Memo

`compose-analysis.md` has five required sections:

1. **Test set design.** What article types you chose. What question types. Why these.
2. **Strategy A results.** Aggregate EM/F1. Where Strategy A wins. Cite specific (qid) examples.
3. **Strategy B results.** Aggregate EM/F1. Where Strategy B wins. Cite specific (qid) examples.
4. **Faithfulness analysis.** At least one example where the summary omitted the answer evidence — Strategy B failed because the summarization step lost the necessary detail. Quote the article, the summary, and the failed answer. This is the section that puts the cost of summarization-as-preprocessing on the page.
5. **Recommendation.** Specific quantitative thresholds, not "it depends." For example: "Use Strategy A (full-article QA with chunking) when article > 800 tokens AND answer location is unknown; use Strategy B (summarize-then-QA) when article > 2,000 tokens AND the question is high-level (top-of-document signal)." Anchor your thresholds in measured numbers from your test set.

The memo should fit on 1.5 pages. The TA rubric rewards quantitative specificity; it penalizes "trade-offs exist" generic conclusions.

Submitting

```
git add qa_test_set.csv pipeline_compose.py compose_predictions.csv compose_metrics.json
git commit -m "Complete stretch thu: summarize-then-qa pipeline"
git push -u origin stretch-thu-summarize-then-qa
```

Open a PR from your stretch branch into `main`. PR description: per-strategy EM/F1 + 1-sentence summary of your recommendation's central threshold.

Paste your PR URL into TalentLMS → Module 7 → Stretch Thu (Honors) to submit this assignment.

CI Autograder

A separate stretch autograder workflow runs on your `stretch-thu-*` branch and on PRs that touch `stretch/thursday/**` — it's a **structural guardrail** (test set schema, `ga_utils.py` present, both strategies implemented in the right order, output schemas, memo sections). Make sure your CI is green before submitting; if it's red, the autograder's failure messages tell you what's structurally wrong with the deliverable.

What's Next

This stretch is your last assignment in Module 7. Submit by Thursday EOD alongside your Integration 7B and Reflection.

Module 8 (T11, Sun 31 May 2026) is RAG — Retrieval Augmented Generation. The QA pipeline you built in Lab 7B becomes the *generator* side of a retrieval system, with Weaviate as the retriever. The summarize-then-QA pattern you just explored is one of the design choices RAG systems wrestle with: do you retrieve full passages, or pre-summarize and retrieve summaries? You will revisit this question in M8.